Roll No: Group:
Name:

CS1201: Data Structure
End Semester Examination (Spring 2025)

Queries will not be addressed during the exam. If any data is missing, please make reasonable assumptions

Q1 (2pt) Let an algorithmic procedure divide the size n problem according to the recurrence relation T(n)=2T(n/2)+O(n). The average case time complexity is.....

Q2 (2pt) What is the asymptotic notation of the following function?

```
int fun(int n, int a[]) {
 s = 1;
 for (i=0; i<n; i++)
 s = s + a[n-1]*a[i]*fun(n-1,a);
 return s;  }
```

Q3 (2pt) Which of the following characterizes a Huffman coding tree?

a. all items are stored at the leaves ; **b.** no nodes have one child; **c.** the tree is balanced ; **d.** (a) and (b) only; **e.** all three of (a), (b), and (c)

Q4 (2 pt) If the shortest path algorithm is run and a vertex is not reachable from the starting point, a distance of ..... is reported

Q5(2 pt) Which traversal computes the total size of each directory in a file system?

Q6: (2 pt) The worst-case runtime complexity of finding the largest item in a min-heap is.....

Q7(2 pt)The minimum number of nodes in a binary tree with L leaves is .....

Q8(2 pt) Consider an array of integers: [4, 10, 3, 5, 1]. What will the array look like after performing the first heapify operation on this array (to build a max-heap)?

Q9(2 pt) The following items are inserted into a binary search tree: 8, 4, 3, 2, 5, 9, 6, 1 ..... placed at a root and ..... node is the deepest?

Q10(2pt) The following items are inserted into an AVL tree: 1, 2, 3, 8, 6. How many rotations are performed? **a.** no rotations **b.** 1 single rotation only **c.** 1 double rotation only **d.** 1 single rotation and 1 double rotation **e.** none of the above

Q11(2 pt) The number of possible min-heaps that can be formed from the values {1,2,3,4,5,6,7} is.....

Q12(2 pts) N elements are inserted one by one into an initially empty binary heap. The total running time (worst case) is .....

Q13(2 pt) Given a 5 element stack S (from top to bottom: 2, 4, 6, 8, 10), and an empty queue Q, remove the elements one-by-one from S and insert them into Q, then remove them one-by-one from Q and re-insert them into S. The content of S is (from top to bottom): .....

Q14(2 pt) Given an efficient circular array-based queue Q capable of holding 10 objects. Show the final contents of the array after the following code is executed:

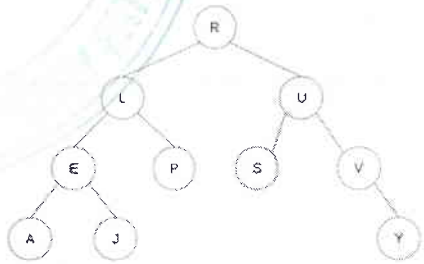
```
for (int k = 1; k ≤ 7; k++)
  Q.enqueue(k);
for (int k = 1; k ≤ 7; k++)
  Q.enqueue(Q.dequeue());
```

Q15 (2 pt) Let A be an array of 31 numbers consisting of a sequence of 0's followed by a sequence of 1's. The problem is to find the smallest index i such that A[i] is 1 by probing the minimum number of locations in A. The worst case number of probes performed by a Binary Search algorithm is.....

Q16 (2 pt) Let G be a graph with n vertices and m edges. What is the tightest upper bound on the running time on Depth First Search of G? Assume that the graph is represented using adjacency matrix.

Q17 (2 pt) The total number of spanning trees (T) in a fully connected graph with 5 vertices is:.....

Q18 (3 pt) Write down the Pre-order, In-Order and Post-Order traversals of the binary tree in the given figure.



Q19(2 pt) Consider Tree in Q18; answer yes or no to indicate whether the tree might represent each of the following data structures

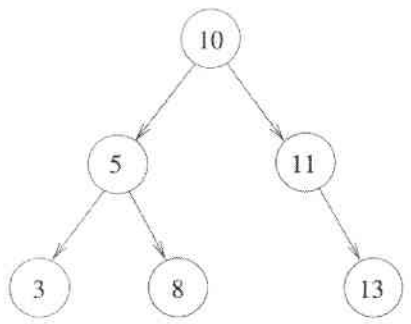
1)	AVL Tree	***YES/NO***
2)	Complete Binary Tree	**YES/NO**
3)	Full Binary Tree	**YES/NO**
4)	Binary Search Tree	**YES/NO**

Q20(3 pt) The following code attempts to delete a node from a BST, but has a logical error. Find and fix it:

```
struct Node* delete(struct Node* root, int key) { if (root == NULL) return NULL;
if (key < root->data) root->left = delete(root->left, key);
else if (key > root->data) root->right = delete(root->right, key);
else {
    if (root->left == NULL) return root->right;
    if (root->right == NULL) return root->left;

    struct Node* temp = root->right;
    while (temp->left != NULL)
        temp = temp->left;

    root->data = temp->data;
    root->right = delete(root->right, root->data);
}
return root;    }
```

Q21 (3 pt) Consider the C function foo and the binary tree shown. When foo is called with a pointer to the root node of the given binary tree, what will it print?

```
typedef struct node {
  int val;
  struct node *left, *right;
} node;

int foo(node *p) {
  int retval;
  if (p == NULL)
    return 0;
  else {
    retval = p->val + foo(p->left) +
foo(p->right);
    printf("%d ", retval);
    return retval;   } }
```



Q22 (3 pt) This code implements deletion of the root from a max heap. Identify the logical error and correct the code

```
void heapify(int heap[], int n, int i) {
  int largest = i;
  int left = 2 * i;
  int right = 2 * i + 1;
  if (left < n && heap[left] > heap[largest])
    largest = left;
  if (right < n && heap[right] > heap[largest])
    largest = right;
  if (largest != i) {
    int temp = heap[i];
    heap[i] = heap[largest];
```

```
void deleteRoot(int heap[], int*
size) {
  if (*size <= 0) return;

  heap[0] = heap[*size - 1];
  (*size)--;
  heapify(heap, *size, 0);
}
```

heap[largest] = temp; heapify(heap, n, largest); } }	

Q23(3 pt) Suppose a Binary Search Tree (BST) is constructed by inserting the keys in the following order: 15, 10, 20, 8, 12, 17, 25, 6, 11, 16, 27. List all the nodes that have exactly one child, and that child is a left child only?

Q24 (3 pt) The following C function inserts a new node into a Binary Search Tree (BST), while avoiding duplicates. Some parts of the code are missing. Complete the code snippet by filling in the blanks.

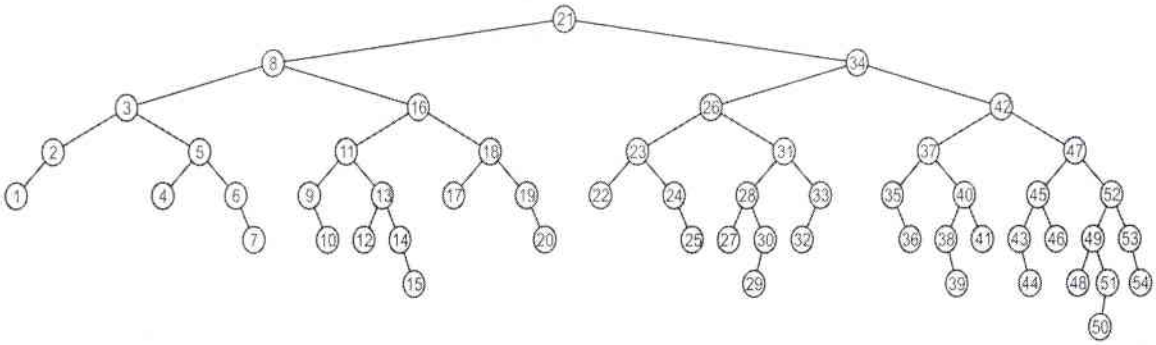
typedef struct node {
int value;
struct node *left, *right;
} Node;
 	
Node* create_node(int value)
{
Node* new_node =
(Node*)malloc(sizeof(Node));
new_node->value = value;
new_node->left =
new_node->right = NULL;
return new_node;
}
	Node* insert(Node* root, int value) {
	if (root == NULL) {
	_____; // Line 1return a new node)
	}
	if (value < root->value) {
	_____; // (Line 2: Insert in the left
	subtree)
	} else if (value > root->value) {
	_____; // (Line 3: Insert in the right
	subtree)
	} else {
	printf("Duplicate value %d not allowed\n", value);
	} return root; }

Q25: (5 pt) Given Pre-order:A B D E G H C F ; In order: D B G E H A C F ; Draw the binary tree .

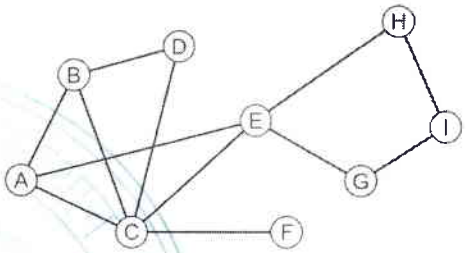
Q26 (9 pt) The following C code snippet handles insertion into a non-full B-Tree node, including splitting a full child node. Some parts are missing. Complete the code by filling in the blanks.

```
#define MAX_KEYS 10  // For simplicity; in real B-Trees, MAX_KEYS = 2*t - 1
typedef struct BTreeNode {
   int keys[MAX_KEYS];
   struct BTreeNode* children[MAX_KEYS + 1];
   int n;     // Current number of keys
   int leaf;  // 1 if node is leaf, 0 otherwise
} BTreeNode;

// Function to split the full child y at index i in parent x
void split_child(BTreeNode* x, int i, int t) {
   BTreeNode* y = x->children[i];
   BTreeNode* z = (BTreeNode*)malloc(sizeof(BTreeNode));
   z->leaf = y->leaf;
   z->n = t - 1;
   for (int j = 0; j < t - 1; j++)
      z->keys[j] = y->keys[j + t];
   if (!y->leaf) {
      for (int j = 0; j < t; j++)
         z->children[j] = y->children[j + t];   }
      x->children[j + 1] = x->children[j];
   x->children[i + 1] = z;
   for (int j = x->n - 1; j >= i; j--)
      x->keys[j + 1] = x->keys[j];
   ________________;  // (Line 1: Move y's median key up to x)
   y->n = t - 1;   x->n++; }
   for (int j = x->n; j >= i + 1; j--)

// Insert key into a non-full node
void insert_non_full(BTreeNode* x, int k, int t) {
   int i = x->n - 1;

   if (x->leaf) {
      while (i >= 0 && k < x->keys[i]) {
         x->keys[i + 1] = x->keys[i];
         i--;
      }
      ________________;  // (Line 2: Insert key into correct position)
      x->n++;
   } else {
      while (i >= 0 && k < x->keys[i])
         i--;
      i++;
      if (x->children[i]->n == 2 * t - 1) {
         split_child(x, i, t);
         if (k > x->keys[i])
            i++;
      }
      ________________;  // (Line 3: Recurse into the appropriate child)
   }
}
```

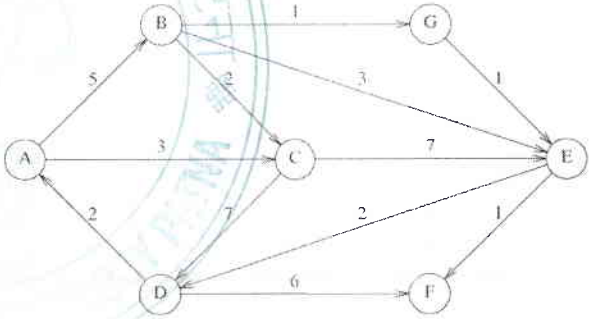
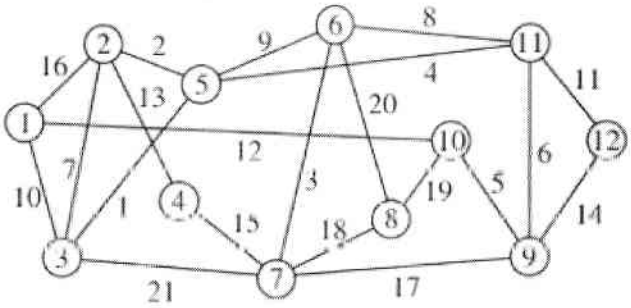
Q27(10 pt) A balanced AVL tree is given below. Let's delete leaf node 1. List the nodes that need balancing post-deletion. Further, obtain the balanced AVL tree post-deletion.



Q28 (5pt) Compare the Depth-First Search (DFS) and Breadth-First Search (BFS) traversals of the graph shown in figure (with starting node A)



Q29(8pt) Using Dijkstra's algorithm, find the shortest path from A to all other vertices for the graph in figure.



Q30(9 pt) Consider the graph in the figure, let V be the set of vertices covered at a given time. Further, we employ Prim's algorithm to construct the minimum spanning tree (MST) with starting vertex 10. Give the order in which the vertices enter V. Also, give the total weight of the obtained MST.